计算机系统基础实验报告

Lab 2：BombLab

姓名：陈璋睿　学号：23300160006　2026 年 10 月 30 日

一、 实验目的与任务

本次实验通过反汇编和 GDB 分析个人二进制炸弹，练习函数调用、循环、分支、递归、结构体与函数指针的阅读方法。任务是在 `defuser.c` 中，根据每次输入的 64 位 `challenge` 计算口令，并完成六个正式关卡和 Honor。

实验文档 [1] 要求恢复随输入变化的计算关系。本报告按“提出猜测、观察中间值、修正解释、换输入验证”的顺序整理调试过程。过程记录保存在父目录 `try.md`；正文选取影响解法的关键指令和实际试验。

二、 实验环境与分析方法

2.1 实验环境

项目	实际环境
系统与架构	Ubuntu 22.04.4 LTS，Docker，Linux x86-64
编译器	GCC 11.4.0，C11，`-O2 -Wall -Wextra -Wpedantic -Werror`
调试与反汇编	GDB 12.1，GNU binutils 2.38
测试与版本管理	Python 3.10.13，Git 2.34.1
个人仓库	`https://github.com/ritachen3656251-design/ICS_lab2`

2.2 从一次运行到通用计算

分析时先固定 seed 42，使用占位口令观察比较位置，再用 seed 43 核对关系。seed 43 位于公开列表之外。GDB 中常用 `break *phase_02`、`ni`、`finish`、`p/u` 和 `x`，其中反汇编与内存读取方法参考 GDB 文档 [3, 4]。

本包中第一参数 `rdi` 通常指向口令字符串，第二参数 `rsi` 保存 challenge。进入下一次函数调用时，再按该次调用重新判断参数。32 位结果用 `uint32_t` 表达，加、减、乘按模 2^{32} 回绕。

三、 正式关卡的分析与尝试

3.1 第 1 关：跟随 helper 的参数和返回值

初步判断与占位尝试

`phase_01` 把 1 传给 `valid_u32_fields`，`sscanf` 后也比较返回值是否为 1，因此口令包含一个整数。先输入 `0`，程序输出 `BOOM!!!`。随后在 `p1_compute` 入口设断点，seed 42 的 challenge 为 `0x28efe333b266f103`；执行 `finish` 后，`eax` 为 1515805119。

核对调用顺序

```
p1_compute:
    shr $0x20,%rax
    shr $0x10,%rdx
    call p1_step2
    ...
    call p1_step0
    ...
    call p1_step1
```

三个 helper 的实际调用次序是 step2、step0、step1。先从 challenge 得到三个 32 位量：

```
low = (uint32_t)challenge;
high = (uint32_t)(challenge >> 32);
middle = (uint32_t)(challenge >> 16);
```

再根据各次调用前的参数寄存器恢复出：

```
a = (5u * high) ^ (low + 0xb15eu);
b = (5u * low + middle) ^ 0x1db3u;
answer = (b + 4u * a) ^ 0xe881u;
```

这里的 middle 是右移后截取的 32 位，包含原 challenge 的第 16 至 47 位。逐次标出参数后，三个函数的关系才清楚；只看 helper 名称很容易把先后顺序写反。

实现与换 seed 验证

将结果交给 `write_answer` 输出。公开测试为 10/10；seed 43 得到 2034676901，实际运行结果见图1。进一步测试覆盖了全零、全一、上下半字和固定随机 challenge。

```
Phase 1 | seed 43

$ ./defuser --seed 43 --phase 1
2034676901

$ ./bomb --seed 43 --phase 1
Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
```

图 1: 第 1 关，seed 43 的实际终端输出渲染

3.2 第 2 关：从首项开始，逐轮恢复数组

第一次：六个零为什么失败

格式检查和 sscanf 都要求六个整数，栈上六项从 rbp-0x40 开始，每项占 4 字节。先输入六个零，在 0x40216a 的比较处观察到：esi=0，eax=14。这说明程序已经读完输入，失败发生在首项比较。

首项比较前只有三步：

```
shr $0xa,%rax
and $0x1f,%eax
add $0x7,%eax
```

右移 10 位后保留低 5 位，再加 7，得到 values[0]=((challenge>>10)&31)+7。seed 42 的首项为 14，seed 43 为 29。只补上首项后，程序能到达循环入口，随后在第二项失败。

第二次：只观察第一轮

```
lea 0x7(%rdx),%ecx
shr %cl,%rax
and $0xf,%eax
add %edx,%eax
xor %esi,%eax
mov -0x40(%rbp,%rdx,4),%esi
lea (%rax,%rax,2),%eax
cmp %eax,%esi
```

第一轮 edx=1，移位量为 8。一个容易读乱的地方是 esi：执行 xor 时它保存前一项，后面的 mov 才把当前输入装进来。lea 中的比例寻址在这里计算三倍。

为核对运算符，设置“把异或读成加法”的对照。seed 42 中 $16 \oplus 14 = 16 + 14 = 30$，两种解释都会算出第二项 90；换 seed 43 后，两者分开了：

seed	前项	片段	片段加 1	异或结果	正确第二项	加法对照
42	14	15	16	30	90	90
43	29	8	9	20	60	114

输入 29 114 0 0 0 0 时，GDB 显示期望为 60、实际输入为 114；改成 29 60 0 0 0 0 后，程序进入下一轮。这个对照是分析时特意设置的错误版本。

第三次：再看第二轮，推广循环

seed 42 在 i=2 时，前项已经变成 90；移位量为 9，取到片段 15，计算

$$(15 + 2) \oplus 90 = 75, \qquad 3 \times 75 = 225.$$

于是恢复出：

```
values[0] = (uint32_t)((challenge >> 10) & 31u) + 7u;
for (unsigned i = 1; i < 6; ++i) {
    uint32_t part = (uint32_t)((challenge >> (i + 7)) & 15u);
    values[i] = (values[i - 1] ^ (part + i)) * 3u;
}
```

循环处理下标 1 至 5，首项加上五次递推恰好得到六项。各轮取出的 4 位片段存在重叠。seed 42 的完整口令为 14 90 225 705 2130 6420；seed 43 得到图2中的结果。公开测试为 10/10。

```
Phase 2 | seed 43

$ ./defuser --seed 43 --phase 2
29 60 150 477 1398 4212

$ ./bomb --seed 43 --phase 2
Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.
```

图 2: 第 2 关，seed 43 的实际终端输出渲染

3.3 第 3 关：索引和 helper 编号的区别

先确定两项输入

`phase_03` 读两个整数，第一项与 `challenge&7` 比较。第二项由下面的参数调用 dispatcher 后得到：

```
shr $0x3,%rax
shr $0x20,%rsi
xor %eax,%esi
call p3_dispatch
```

令 `index=challenge&7`，`x=(uint32_t)(challenge>>3) ^ (uint32_t)(challenge>>32)`。

读取跳转表并做对照

dispatcher 中的关键指令是：

```
cmp $0x7,%eax
ja  <out_of_range>
notrack jmp *0x405820(,%rax,8)
```

用 `x/8gx 0x405820` 读取八项地址，再跟到各地址的跳转目标。表的实际顺序为 case6、case7、case2、case0、case1、case5、case3、case4。

用 challenge 0 检验“索引 0 对应 case0”的解释：它会给出 `0 19859`，实际运行失败。GDB 中 dispatcher 返回 34764，按表映射得到的 `0 34764` 则通过。函数编号和输入索引在这份炸弹中有一层映射。

恢复八个分支

索引	目标	第二项的计算
0	case6	`(x ^ 0x87ccu) + (x >> 2)`
1	case7	`5u * x + 0x49160u`
2	case2	`5u * x + 0x9a24u`
3	case0	`(x ^ 0x4d93u) + (x >> 2)`
4	case1	`5u * x + 0xb829u`
5	case5	`3u * x + 0xfe6au`
6	case3	`(4u * x) ^ 0x833du`
7	case4	`(2u * x) ^ 0x1105u`

实现保留了八个显式 case，便于和表逐项对应。公开测试为 10/10，另用八种索引及带高位数据的样本核对了各分支。seed 43 的口令为 7 3149956987。

```
Phase 3 | seed 43

$ ./defuser --seed 43 --phase 3
7 3149956987

$ ./bomb --seed 43 --phase 3
Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.
```

图 3: 第 3 关，seed 43 的实际终端输出渲染

3.4 第 4 关：从基本情况到递归回溯

第一次：先让目标恰好等于中点

本关读一个整数。p4_walk 的参数为：

```
target = challenge & 127;
low = 0; high = 127;
salt = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32);
```

取 challenge 63，目标正好等于首次中点，salt 为 0。先输入目标值 63，结果失败；在函数入口执行 finish 后，eax=311，对应汇编：

```
lea 0xf8(%rdi),%eax
xor %ecx,%eax
ret
```

因此基本情况返回 (mid+248)^salt。补成 311 后，这个样本通过。

第二次：只走一次左分支

取 challenge 31，调用区间依次为 $[0, 127]$、$[0, 62]$。子调用命中中点，第一次 finish 看到返回 279；回到父调用再执行 finish，结果为 886。父调用返回前的指令为：

```
call p4_walk
lea (%rax,%rax,2),%eax
xor %ebx,%eax
...
xor $0xc,%eax
```

其中 ebx=mid+salt。右分支的最后一个常量是 0xa。这说明递归的路径影响回溯时的加工过程。

递归关系与验证

令 $m = l + \lfloor (h - l)/2 \rfloor$，则返回关系可写为：

```
if (target == mid)
    return (mid + 248u) ^ salt;
if (target < mid) {
    value = phase4_walk(target, low, mid - 1u, salt);
    return (3u * value) ^ (mid + salt) ^ 12u;
}
value = phase4_walk(target, mid + 1u, high, salt);
return (3u * value) ^ (mid + salt) ^ 10u;
```

实现保持递归形式，与 GDB 调用栈对应。目标范围为 0 至 127，测试额外覆盖全部 128 种目标；公开测试为 10/10。seed 43 返回 3949797497。

```
Phase 4 | seed 43

$ ./defuser --seed 43 --phase 4
3949797497

$ ./bomb --seed 43 --phase 4
Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
```

图 4: 第 4 关，seed 43 的实际终端输出渲染

3.5 第 5 关：先处理头节点，再沿 next 遍历

第一次读取时的宽度问题

本关读两个整数，`p5_traverse` 通过 `eax` 返回第一项，通过第二个指针参数写回另一项。读取起点表时，开始用了 `x/8uw`，输出中出现了指针低半部和一串零；回看索引比例 `8`，才发现这只读了四个 64 位指针。改成 `x/8gx` 后，完整八项起点对应节点下标

$$5,\ 8,\ 3,\ 1,\ 9,\ 11,\ 0,\ 7.$$

由访问偏移恢复布局

```
mov 0x405b60(,%rax,8),%rdx
mov 0x8(%rdx),%esi
add 0xc(%rdx),%eax
...
mov (%rdx),%rdx
```

节点的 +0 处是 8 字节 next 指针，+8 是 4 字节 id，+12 是 4 字节 bias，每个节点共 16 字节。按 next 指针得到的完整链为：

```
5 -> 8 -> 3 -> 1 -> 9 -> 11 -> 0 -> 7 -> 10 -> 2 -> 6 -> 4 -> NULL
```

静态节点的存储顺序和遍历顺序由此区分开来。

第二次：用两个相邻 challenge 核对处理次数

challenge 0 选中起点 nodes[5]，额外跳转数为 0。观察到节点字段为 id=1049、bias=5848，函数仍先处理它，得到 `95042 1049`。将 challenge 改为 8，起点相同、额外跳转数变成 1。沿用

原答案会失败；单步执行 `mov (%rdx),%rdx` 后，指针进入 nodes[8]，字段为 id=55、bias=37381，最终得到 `322556 55`。额外跳转数和总处理数相差一轮。

恢复遍历关系

```
value = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
value ^= 0x7581u;
count = ((challenge >> 3) & 3u) + 1u;
for (unsigned i = 0; i < count; ++i) {
    value = (3u * value + node->bias) ^ node->id;
    last = node->id;
    node = node->next;
}
```

输出为累计值和最后处理节点的 id。代码中的静态节点表来自本包数据，运行时仍依据 challenge 选择起点和长度。公开测试为 10/10，另覆盖八个起点与四种长度的全部组合。seed 43 的结果见图5。

```
Phase 5 | seed 43

$ ./defuser --seed 43 --phase 5
3601218346 1313

$ ./bomb --seed 43 --phase 5
Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.
```

图 5: 第 5 关，seed 43 的实际终端输出渲染

3.6 第 6 关：两次间接调用之间的数据传递

先纠正输入数量和表长

沿用上一关的 `0 0` 试探输入时，程序还没到 `p6_dispatch` 就失败。回看入口，格式检查要求一个整数。改为 `0` 后，能停在 dispatcher 的第一次调用处。

另一个起初读少的地方是函数表：虽然只有六个不同的 op 函数，索引却取三位，表中实际有八项。用 `x/8gx 0x405e00` 读到的顺序为 `op1, op5, op4, op0, op4, op3, op2, op3`。

区分循环移位与普通右移

取 challenge 67，第一次索引为 3，参数为 1，间接调用落到 `p6_op_0`。执行 `finish` 得到 `0x80000000`，由此核对 `ror` 把低位补回到高位。六个操作恢复如下：

函数	32 位计算关系
op0	`(x >> 1) | (x << 31)`
op1	`(2u*x + 0xc3ecu) ^ (7u*x)`
op2	`(x ^ (x >> 7)) + 0xe4a9u`
op3	`(x + 0xca25u) ^ (x >> 2)`
op4	`((x << 2) | (x >> 30)) ^ 0x62e5u`
op5	`(x ^ 0xf6bcu) + (x << 7)`

再跟第二次调用

```
call *0x405e00(,%rax,8)
xor $0x8322,%eax
mov %eax,%edi
call *0x405e00(%rbx)
add $0x671c,%eax
```

challenge 0 的第一返回值是 50156，直接输入它会失败。继续观察，第二函数接收 `0x40ce`，返回 32810；最后再加 `0x671c`，得到 59206。因此完整顺序为：

```
x = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
x = ops[challenge & 7u](x);
x ^= 0x8322u;
x = ops[(challenge >> 3) & 7u](x);
answer = x + 0x671cu;
```

实现使用六个小函数和八项函数指针表。公开测试为 10/10，额外覆盖八乘八索引组合及高低位旋转样本。seed 43 输出 591440785。

```
Phase 6 | seed 43

$ ./defuser --seed 43 --phase 6
591440785

$ ./bomb --seed 43 --phase 6
Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.
```

图 6: 第 6 关，seed 43 的实际终端输出渲染

四、 Honor：从字符串布局到三个整数

4.1 先进入 secret_gate，再确定标志位置

把六关正确口令作为前缀，再附加一行 `test`，可以停在 `secret_gate`。seed 42 下观察到：

```
challenge = 0xccf635ee9e9e2fa4
context   = 0x5d8b1809d64489a0
```

此时函数返回 0。先从长度检查和三个栈地址看起：

```
sub $0x24,%rax
cmp $0x7f,%rax
jbe <copy>
...
lea -0xd0(%rbp),%r12
...
cmpl $0x4e45504f,-0xb0(%rbp)
lea -0xac(%rbp),%rdi
call secret_phase
```

`strlen-36` 的无符号比较给出长度区间 36 至 163。复制起点到标志位置相距 32 字节，标志到数字子串又相距 4 字节。小端内存中的 `0x4e45504f` 对应字符串 `OPEN`，布局为：

相对起点偏移	长度	内容
0–31	32 字节	填充字符 A
32–35	4 字节	OPEN
36 起	随数字长度变化	三个无符号十进制整数

用 31 个 A 加 `OPEN0 0 0` 做偏移对照，检查处读到 `0x304e4550`，即错位的 `PEN0`；换成 32 个 A 后，检查值成为 `0x4e45504f`，并进入 `secret_phase`，其 `rdi` 指向的内容恰为 `0 0 0`。这个试验把标志检查和数字检查分开了。

4.2 只修正第二个整数，再定位第三个

数字检查先将 `context&3` 与第一项比较。seed 42 对应 0，所以三个零已经能通过这一项。继续停在 `0x4026ec`，观察第二项比较。中间 helper 为：

```
secret_mix:
    mov %edi,%eax
    ror $0x5,%eax
    xor $0xf159,%eax
    ret
```

将 challenge 与 context 分别按高低 32 位异或折叠为 a, b，令 $m = \text{context} \,\&\, 3$，得到：

```
mode = (uint32_t)(context & 3u);
a = (uint32_t)challenge ^ (uint32_t)(challenge >> 32);
b = (uint32_t)context ^ (uint32_t)(context >> 32);
x = a ^ b;
x = (x >> 5) | (x << 27);
first = x ^ 0xf159u ^ (uint32_t)(context >> 16)
        ^ (mode * 0x6a9eu);
second = first ^ (b + mode + 0x9bc7u + (a >> 5));
```

GDB 观察到 $a = 1382554186$、$b = 2345636265$，期望第二项为 113515330。先只填 `OPEN0 113515330 0`，程序继续失败；这次能到达第三项比较 `0x402700`，看到期望值 2292675840。补上后，seed 42 正常输出 `Secret phase defused.`。

这里最容易漏读的是 `context>>16`：随后截为 32 位，使用的是原 context 的第 16 至 47 位。第二、三项的异或和求和也都按 32 位计算。

4.3 通用输出与换 seed 核对

最终将 32 个 A、`OPEN` 和三个动态整数拼成一行。seed 43 的 mode 为 1，生成的第七行为：

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN1 2319039455 4233664519
```

图7来自六关加 Honor 的一次完整运行。Honor 公开测试为 10/10；额外 20 个公开列表之外的 seed 全部通过，并覆盖 mode 0、1、2、3。

Honor | seed 43

```
$ { ./defuser --seed 43; ./defuser --secret --seed 43; } | ./bomb --seed 43
Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.
Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.
Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.
Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
```

图 7: 六关及 Honor，seed 43 的实际终端输出渲染

五、 测试结果与过程核对

5.1 公开测试与列表外验证

本报告整理时重新运行 `python3 public_test.py`，六关为 60/60，Honor 为 10/10，见图8。各节的 seed 43 运行图对应同一份最终 `defuser.c`。

关卡	公开测试	额外覆盖的主要情况
1	10/10	全零、全一、上下半字及随机 challenge
2	10/10	首项位段、逐轮移位、额外 seed
3	10/10	八种索引和带高位数据的分支输入
4	10/10	0 至 127 的全部目标、左右递归
5	10/10	八个起点与四种处理长度
6	10/10	八乘八函数索引、循环移位的补回位
Honor	10/10	额外 20 个 seed、四种 mode

Honor 额外 seed 为 2、3、4、5、6、8、9、10、11、12、13、14、15、16、17、18、43、3141592653、9223372036854775808 和 18446744073709551614。这些输入均不在公开列表中，实际输出行长为 57 至 59 字符。

Public tests | captured summary

```
$ python3 public_test.py
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
[exit status: 0]
```

图 8: 公开测试的统计行与 Honor 明细，实际终端输出渲染

图中的文字由实际执行命令的输出渲染；统计图保留了六关总数和 Honor 明细。各关的完整过程输出保存在父目录 `try.md`。

5.2 环境问题与复编方法

初次直接在 `/oss-ipec` 目录链接时，出现 `file truncated`。把同一份源码、编译器和选项改为向 `/tmp` 输出后，链接成功；完成的 ELF 文件顺序复制回 `BombLab/defuser` 后也能运行。结合 `df -T` 中 ossfs2 与 ext4 的差别，本次后续复编使用临时目录生成二进制的方式。

此外，领取脚本最初遭遇 `ConnectionResetError`。现有代理下，curl 可以下载附件，随后通过原脚本的 `--file` 入口完成导入。这两类问题分别发生在构建和下载阶段，与口令公式的核对分开处理。

5.3 按关记录的提交

提交	对应内容
`0034180`	第 1 关 helper 调用计算
`de0df4a`	第 2 关数组递推
`09ce9eb`	第 3 关跳转表分支
`b965831`	第 4 关递归回溯
`0945dd7`	第 5 关链表遍历
`49baacd`	第 6 关间接调用
`b9596a8`	Honor 字符串与上下文计算

六、 实验体会

第二关给我的印象最深。开始只看到一个错误结果时，很难判断是格式、首项还是循环出了问题。将它拆成“六个零、只补首项、只补第二项、再看下一轮”后，每次失败都对应了更具体的位置。seed 42 中加法与异或的结果碰巧相等，也让我体会到换输入核对中间值的作用。

后面几关沿用这一做法：第三关先读实际跳转地址，第四关分开看子调用与父调用的返回，第五关沿指针而非数组顺序走，第六关跟清两次调用之间的参数，Honor 则先确认字符串布局再算整数。恢复公式时，把位宽、运算顺序和寄存器在当前指令处的含义写清楚，比直接照着某个样本值猜口令更有帮助。

七、 参考资料

[1] 课程实验文档：Lab2 BombLab。`https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/`，访问日期：2026 年 10 月 30 日。

[2] 实验仓库中的 `README.md`、`tutor/README.md`、`teaching/README.md`，以及公开测试的 seed 列表。

[3] GNU GDB Manual：Source and Machine Code。`https://sourceware.org/gdb/current/onlinedocs/gdb.html/Machine-Code.html`。

[4] GNU GDB Manual：Examining Memory。`https://sourceware.org/gdb/current/onlinedocs/gdb.html/Memory.html`。

八、 AI 使用说明